LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – ENKAPSULASI



NAMA: Anindya Sukma Dwiyanda

NIM: 103132400006

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Praktikum ini mempelajari penggunaan modifier untuk mengatur hak akses atribut dan kelas, pemanfaatan constructor dan constructor overloading untuk menginisialisasi objek secara fleksibel, serta penerapan package untuk mengelompokkan struktur kelas agar lebih terorganisir.

B. Guided

1. Guided 1- Manusia.java

```
package project.guided1.Manusia;

public class Manusia {
    // definisi atribut
    private String nama;
    private int umur;

    // 4.2 contructor
    public Manusia(){};
    public Manusia(String nama){
        this.nama = nama;
    }
    public Manusia(String nama, int umur){
        this.nama = nama;
        this.umur = umur;
    }
    //method setter
    public void setNama(String a){
        nama = a;
    }

    public void setUmur(int a){
        this.umur = umur;
    }

    //method getter
    public String getNama(){
        return nama;
    }

    public int getUmur(){
        return umur;
    }
}
```

2. Guided 2 – DemoManusia.java

```
package project.guided1.main;
```

```
import project.guided1.Manusia.Manusia;

public class DemoManusia {
   public static void main(String[] args){
      Manusia arrMns[] = new Manusia[3];

      Manusia objMns1 = new Manusia();

      // kedua
      Manusia objMns2 = new Manusia("John");

      // ketiga
      Manusia objMns3 = new Manusia("Sean", 44);

      arrMns[0] = objMns1;
      arrMns[1] = objMns2;
      arrMns[2] = objMns3;

      for (int i =0; i<3; i++) {
         System.out.println("Nama: " + arrMns[i].getNama());
         System.out.println("Umur: " + arrMns[i].getUmur());
         System.out.println();
      }
   }
}
```

Output :

```
Nama: null
Umur: 0

Nama: John
Umur: 0

Nama: Sean
Umur: 44
```

3. Guided 3 – HargaPulsa.java

```
package project.guided2.HargaPulsa;
public class HargaPulsa {
   public void info(){
      System.out.println("Harga Pulsa");
      System.out.println("Rp. 5000");
      System.out.println("Rp. 10000");
      System.out.println("Rp. 20000");
      System.out.println("Rp. 50000");
      System.out.println("Rp. 100000");
   }
```

```
}
```

4. Guided 4 – HargaToken.java

```
package project.guided2.HargaToken;
public class HargaToken {
    public void info(){
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

5. Guided 5 – Main.java

```
package project.guided2.main;

import project.guided2.HargaToken.HargaToken;
import project.guided2.HargaPulsa.HargaPulsa;

public class Main {
    public static void main(String[] args){
        HargaToken objectToken = new HargaToken();
        objectToken.info();

        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}
```

Output:

```
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
```

C. Unguided

1. Unguided 1- Dataset.java

   Class `Dataset` (package `model`)

   Atribut (implementasikan enkapsulasi):

Atribut	Tipe	Keterangan
`nama`	`String`	Nama dataset
`jumlahBaris`	`int`	Banyak baris (observasi)
`jumlahKolom`	`int`	Banyak kolom (fitur)
`jumlahMissing`	`int`	Banyak sel yang kosong

   Ketentuan tambahan:

   - Buat konstanta `BATAS_MISSING` bernilai `5.0` (persen). Dataset dengan missing diatas batas ini dianggap perlu dibersihkan.
   - Buat variabel milik class yang menghitung total objek `Dataset` yang telah dibuat, beserta method untuk ambil nilainya tanpa perlu membuat objek.

   Constructor (ada 3, overloading):
   1. Tanpa parameter
   2. Hanya parameter `nama`
   3. Lengkap: `nama`, `jumlahBaris`, `jumlahKolom`, `jumlahMissing`

   Setiap constructor harus menambah hitungan total dataset.

   Method:
   - Getter dan setter untuk semua atribut.
   - Pada setter `jumlahBaris`, `jumlahKolom`, dan `jumlahMissing`, buat agar nilai tidak berubah.
   - `getPersentaseMissing()` mengembalikan `double`: persentase sel kosong dibanding total sel (`baris × kolom`). Jika total sel = 0, kembalikan `0`.
   - `perluDibersihkan()` mengembalikan `boolean`: `true` jika persentase missing melebihi `BATAS_MISSING`.

   Jawaban :

```
package unguided.model;

public class Dataset {
    private String nama;
    private int jumlahBaris;
```

```
    private int jumlahKolom;
    private int jumlahMissing;

    public static final double BATAS_MISSING = 5.0;
    private static int totalDataset = 0;

    public Dataset() {
        this.nama = "Tidak diketahui";
        this.jumlahBaris = 0;
        this.jumlahKolom = 0;
        this.jumlahMissing = 0;
        totalDataset++;
    }

    public Dataset(String nama) {
        this.nama = nama;
        this.jumlahBaris = 0;
        this.jumlahKolom = 0;
        this.jumlahMissing = 0;
        totalDataset++;
    }

    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        this.jumlahBaris = jumlahBaris;
        this.jumlahKolom = jumlahKolom;
        this.jumlahMissing = jumlahMissing;
        totalDataset++;
    }

    public static int getTotalDataset() {
        return totalDataset;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
```

```
    }

    public int getJumlahKolom() {
        return jumlahKolom;
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0.0;
        }
        return ((double) jumlahMissing / totalSel) * 100.0;
    }

    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

Output :

```
=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0,00%)
Status       : Bersih
```

Penjelasan :

Bertindak sebagai cetak biru utama untuk merepresentasikan data dataset. Kelas ini menerapkan prinsip enkapsulasi dengan menyembunyikan atribut menggunakan modifier private, menyediakan constructor overloading untuk fleksibilitas pembuatan objek, serta menggunakan getter, setter, dan method logika bisnis untuk mengelola data secara aman.

2. Unguided 2 - LaporanDataset
   Buat method `cetak` yang menerima satu objek `Dataset` dan menampilkan laporannya dengan format:

   ```
   === Laporan Dataset ===
   Nama         : Titanic
   Jumlah Baris : 891
   Jumlah Kolom : 12
   Missing      : 866 sel (8.10%)
   Status       : Perlu dibersihkan
   ```

   Status berisi `Perlu dibersihkan` atau `Bersih` sesuai hasil dari `perluDibersihkan()`.
   Jawaban :

```
package unguided.laporan;

import unguided.model.Dataset;

public class LaporanDataset {
    public static void cetak(Dataset dataset) {
        String status = dataset.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + dataset.getNama());
        System.out.println("Jumlah Baris : " + dataset.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + dataset.getJumlahKolom());
        System.out.printf("Missing      : %d sel (%.2f%%)\n",
dataset.getJumlahMissing(), dataset.getPersentaseMissing());
        System.out.println("Status       : " + status);
        System.out.println();
    }
}
```

   Output :

```
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8,10%)
Status       : Perlu dibersihkan
```

   Penjelasan :

Berfungsi sebagai kelas utilitas khusus penampil informasi yang memisahkan logika tampilan (presentation layer) dari logika data. Kelas ini memiliki method statis untuk menerima objek Dataset dan mencetak rincian laporannya ke konsol dengan format yang rapi dan terstruktur.

3. Unguided 3 - Main
   Buat tiga objek, masing-masing dengan constructor berbeda:

Objek	Cara pembuatan	Data
1	Constructor tanpa parameter, lalu isi memakai setter	Titanic, 891 baris, 12 kolom, 866 missing
2	Constructor 1 parameter	Wine Quality
3	Constructor lengkap	Iris, 150 baris, 5 kolom, 0 missing

   Simpan ketiganya dalam array, cetak laporannya dengan perulangan, lalu tampilkan di akhir:

   ```
   Total dataset dibuat : 3
   ```

   Jawaban :

```
package unguided.main;

import unguided.model.Dataset;
import unguided.laporan.LaporanDataset;

public class Main {
    public static void main(String[] args) {
        Dataset dataset1 = new Dataset();
        dataset1.setNama("Titanic");
        dataset1.setJumlahBaris(891);
        dataset1.setJumlahKolom(12);
        dataset1.setJumlahMissing(866);

        Dataset dataset2 = new Dataset("Wine Quality");

        Dataset dataset3 = new Dataset("Iris", 150, 5, 0);

        Dataset[] daftarDataset = {dataset1, dataset2, dataset3};

        for (Dataset d : daftarDataset) {
            LaporanDataset.cetak(d);
        }
```

```
            System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
        }
}
```

Output :

```
=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0,00%)
Status       : Bersih
```

Penjelasan :

Berperan sebagai titik masuk utama (entry point) yang mengeksekusi seluruh alur program. Di kelas inilah objek-objek dataset diinisialisasi menggunakan berbagai variasi constructor, dikelola dalam bentuk array, lalu diproses bersama kelas laporan untuk menampilkan hasil akhirnya ke layer.

D. Kesimpulan

Praktikum ini berhasil mengimplementasikan konsep enkapsulasi, constructor, modifier, dan package pada studi kasus pencatatan dataset. Melalui pembuatan class Dataset (menggunakan private modifier serta constructor overloading), class LaporanDataset (sebagai utilitas cetak), dan class Main (sebagai eksekusi program), seluruh data berhasil dikelola, divalidasi, dan diorganisasikan ke dalam struktur direktori package yang rapi dan terstruktur sesuai standar Java.